

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier.

# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

print("--- First 5 Rows ---")
print(df.head())

print("--- Basic Info ---")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column, and their data t
# Notice that 'Age' and 'Cabin' have missing values!
df.info()
```

```
--- First 5 Rows ---
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name                               Sex  Age  SibSp \
0  Braund, Mr. Owen Harris             male  22.0     1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0     1
2    Heikkinen, Miss. Laina             female  26.0     0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0     1
4    Allen, Mr. William Henry             male  35.0     0

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0   113803   53.1000  C123      S
4      0  373450   8.0500   NaN      S

--- Basic Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0  PassengerId  891 non-null    int64
1  Survived     891 non-null    int64
2  Pclass       891 non-null    int64
3  Name         891 non-null    object
4  Sex          891 non-null    object
5  Age          714 non-null    float64
6  SibSp        891 non-null    int64
```

```

7  Parch      891 non-null  int64
8  Ticket     891 non-null  object
9   Fare      891 non-null  float64
10 Cabin      204 non-null  object
11 Embarked   889 non-null  object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data.

- ✓ The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())
print("--- Key Insights ---")
```

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on**

- ✓ **Plotting Libraries:**
 - * **Matplotlib:** The foundational library, gives you full control over everything.
 - * **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

- ✓ Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

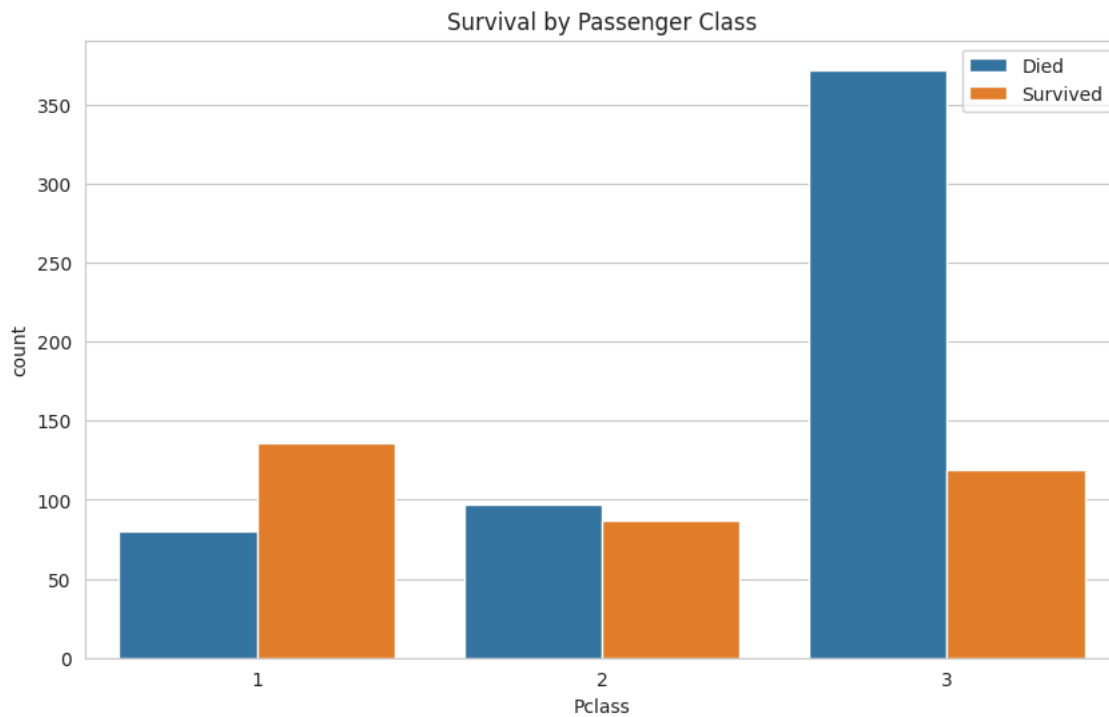
```
sns.set_style('whitegrid') # Sets a nice visual style for our plots
plt.figure(figsize=(8, 6))
sns.countplot(x='Survived', data=df)
```

- ✓ Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

```

plt.figure(figsize=(10, 6))
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class")

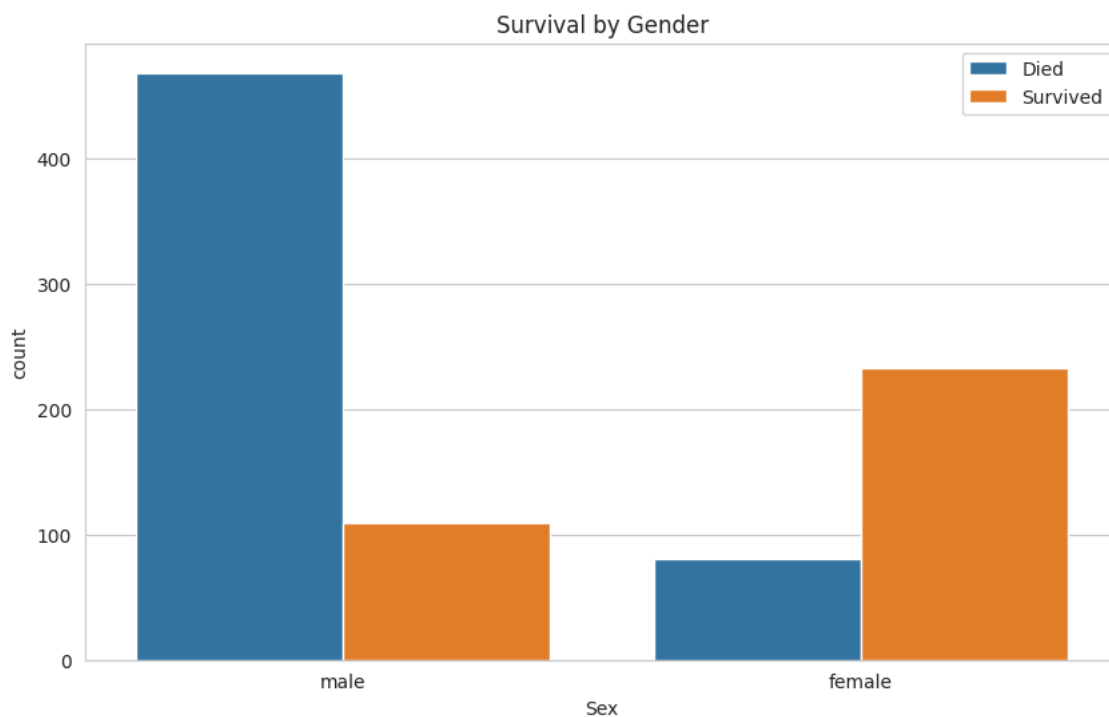
```



Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers.

Visualization 3: What about gender? The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong pattern.")
```



Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong pattern.

Visualization 4: How does age play a role? For a continuous variable like `Age`, a histogram is a great way to see its distribution.

A FacetGrid allows us to create multiple plots side-by-side to compare distributions. # Here, we create one histogram for pass

Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

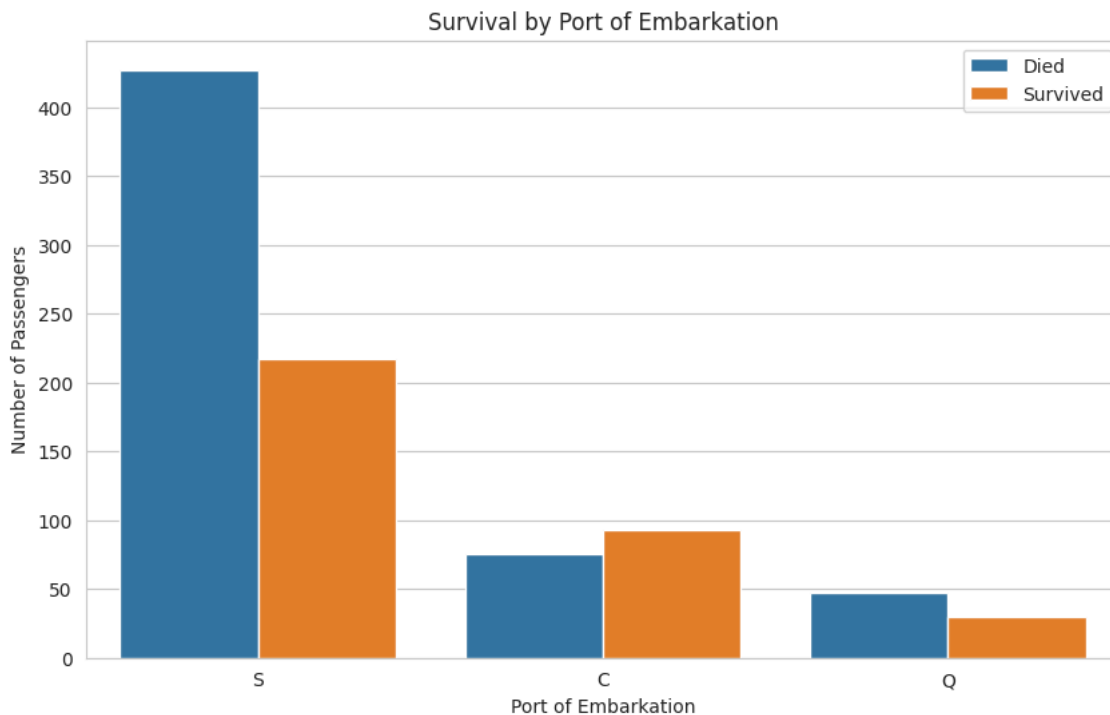
Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C = Cherbourg, Q = Queenstown, S = Southampton).2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

Experiment 1: Port of Embarkation vs. Survival

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation')
plt.xlabel('Port of Embarkation')
plt.ylabel('Number of Passengers')
plt.legend(['Died', 'Survived'])
plt.show()
```

```
print("--- Survival Rate by Port of Embarkation ---")
print(pd.crosstab(df['Embarked'], df['Survived'], normalize='index'))
```

print("\nInsight: The port where passengers boarded does seem related to survival. Passengers who boarded at Cherbourg (C) had



```
--- Survival Rate by Port of Embarkation ---
Survived      0      1
Embarked
C      0.446429  0.553571
Q      0.610390  0.389610
S      0.663043  0.336957
```

Insight: The port where passengers boarded does seem related to survival. Passengers who boarded at Cherbourg (C) had a higher s

Experiment 2: Fare vs. Survival1. `Fare` is a continuous numerical variable.2. A `boxplot` or `violinplot` is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these

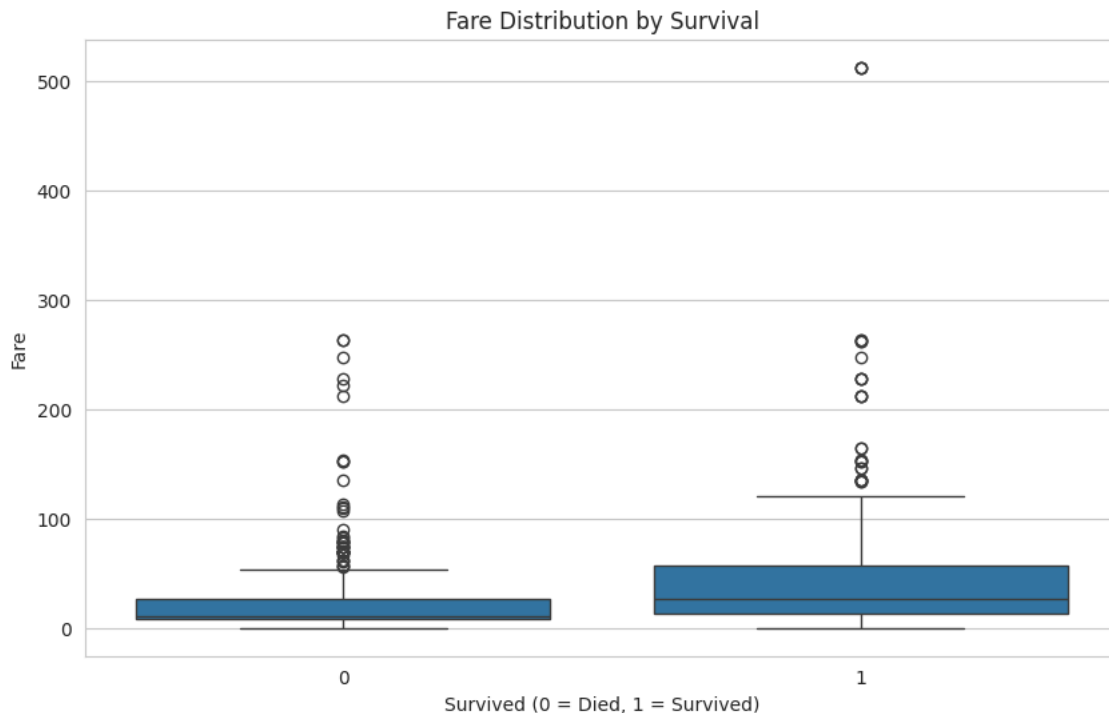
plots to compare the **Fare** distribution by **Survived**. What does it tell you?

```
# Experiment 2: Fare vs. Survival
```

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival')
plt.xlabel('Survived (0 = Died, 1 = Survived)')
plt.ylabel('Fare')
plt.show()
```

```
print("--- Average Fare by Survival Status ---")
print(df.groupby('Survived')['Fare'].mean())
```

```
print("\nInsight: Passengers who survived paid a higher average fare than passengers who did not survive. This suggests that fa
```



```
--- Average Fare by Survival Status ---
Survived
0      22.117887
1      48.395408
Name: Fare, dtype: float64
```

```
Insight: Passengers who survived paid a higher average fare than passengers who did not survive. This suggests that fare may be
```

 **Knowledge Check**Instructions: Answer the following questions in this markdown cell.

1. **What is the primary goal of Exploratory Data Analysis (EDA)?** The primary goal of Exploratory Data Analysis is to understand the dataset before building a model. EDA helps us find patterns, missing values, unusual data, and relationships between variables.
2. **Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic?** (Describe them in terms of class, gender, and age). Based on the plots, the person with the best chance of surviving was likely a female passenger, especially someone in first class. Children also appeared to have a better chance of surviving than many adults. Overall, women, children, and higher-class passengers had the strongest survival patterns.

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier. # Load the dataset directly from a
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
print("--- First 5 Rows ---")
print(df.head())
print("--- Basic Info ---")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column, and their data t
```

[Show hidden output](#)

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data.

The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())
print("--- Key Insights from Statistics ---")
print(f"The average age of a passenger was {df['Age'].mean():.1f} years.")
print(f"The overall survival rate was {df['Survived'].mean():.1%}.")
print(f"Fares ranged from ${df['Fare'].min()} to a whopping ${df['Fare'].max()}."
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

--- Key Insights from Statistics ---

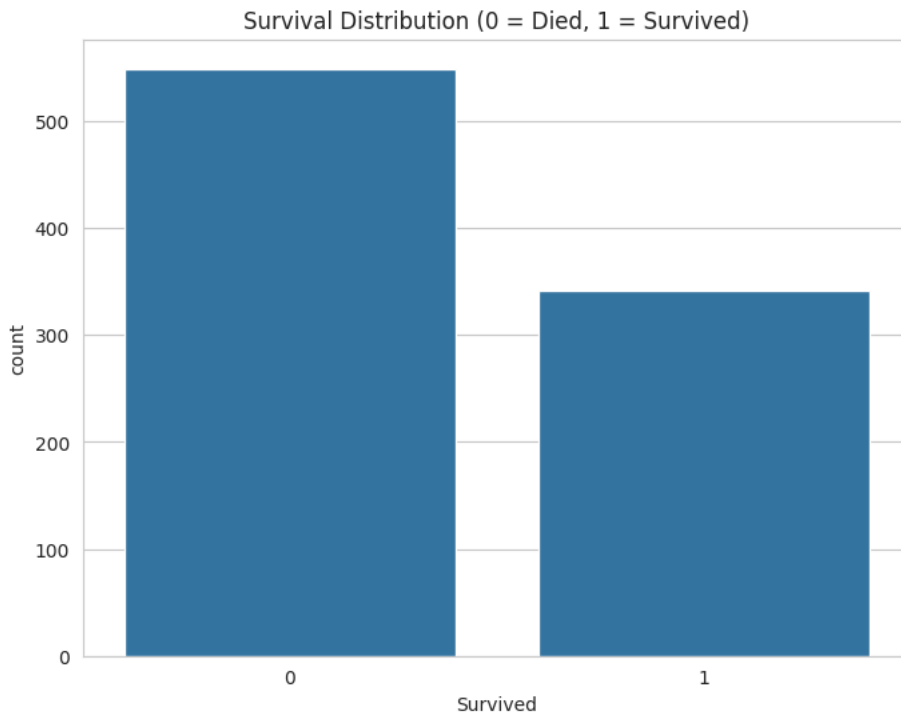
The average age of a passenger was 29.7 years.
 The overall survival rate was 38.4%.
 Fares ranged from \$0.0 to a whopping \$512.3292.

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on**

- Plotting Libraries:
 - * Matplotlib:** The foundational library, gives you full control over everything.
 - Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

- Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

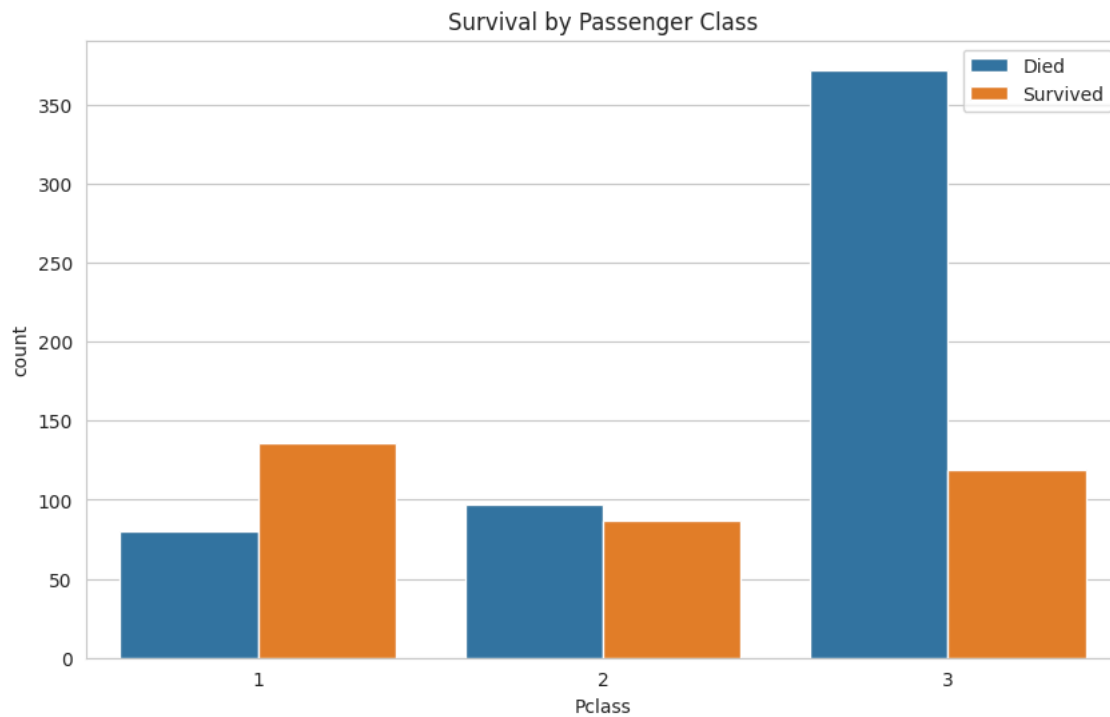
```
sns.set_style('whitegrid') # Sets a nice visual style for our
plt.figure(figsize=(8, 6))
#This shows amount that survived
sns.countplot(x='Survived', data=df)
plt.title('Survival Distribution (0 = Died, 1 = Survived)')
plt.show()
print("Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes be a challenge for
```



Insight: Far more people died than survived. This is an example of an imbalanced dataset, which can sometimes be a challenge for

- Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

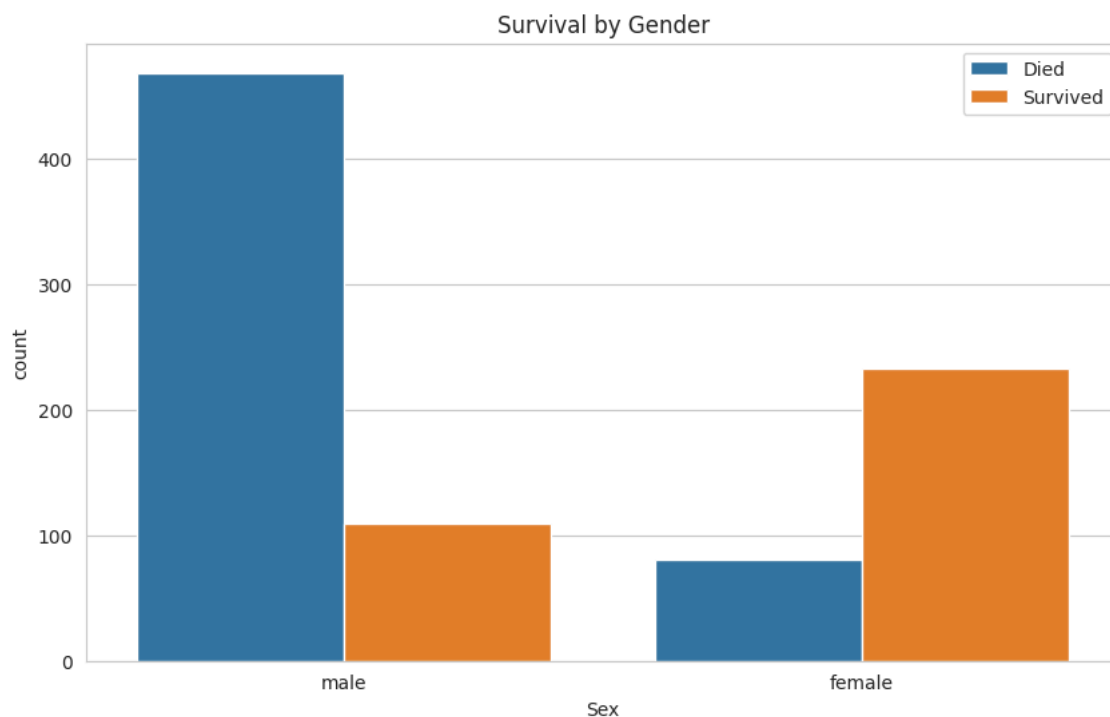
```
plt.figure(figsize=(10, 6))
#Shows the class and shows the survivors from the dataSet
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers")
```



Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers.

Visualization 3: What about gender? The 'women and children first' mantra is famous. Let's see if the data supports it.

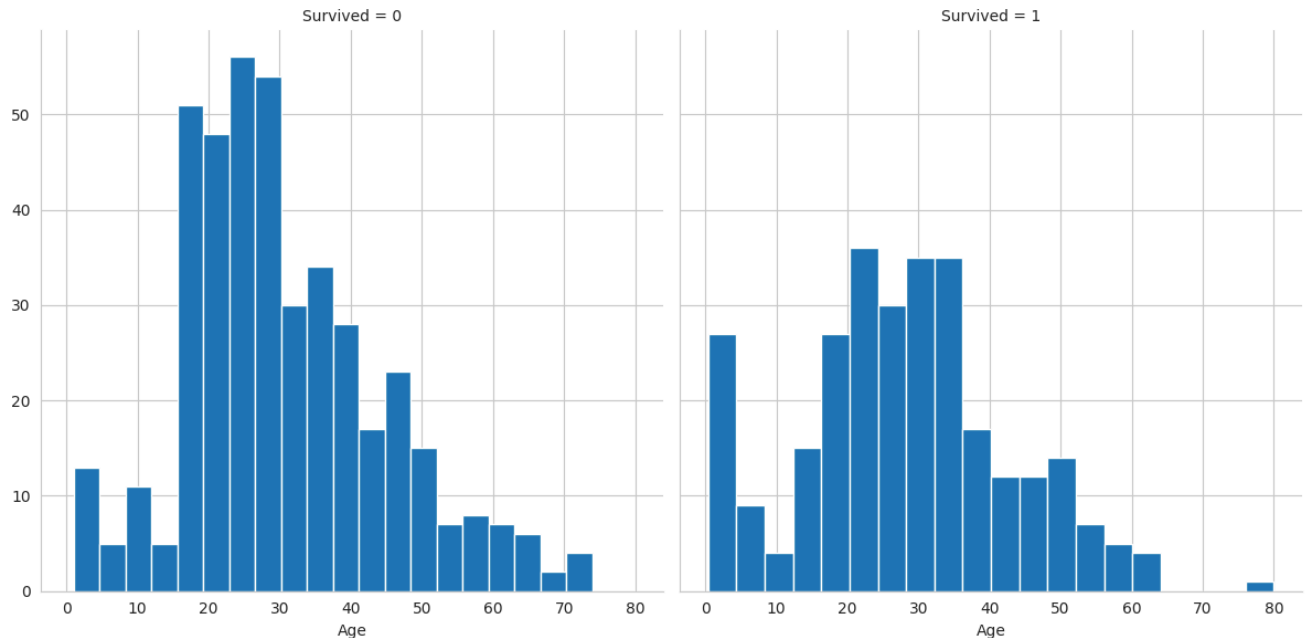
```
plt.figure(figsize=(10, 6))
#Shows the genders that were more or less likely to survive
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong pattern.")
```



Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another very strong pattern.

- Visualization 4: How does age play a role? For a continuous variable like `Age`, a histogram is a great way to see its distribution.

```
# A FacetGrid allows us to create multiple plots side-by-side to compare distributions.
# Here, we create one histogram for passengers who died (col='Survived'=0) and one for those who survived (col='Survived'=1).
g = sns.FacetGrid(df, col='Survived', height=6)
#Shows a histogram of who survived and who died and there age
g.map(plt.hist, 'Age', bins=20)
plt.show()
print("Insight: The age distribution for those who did not survive is centered around the 20-40 age range. For those who survived, there is a wider distribution centered around the 20-40 age range.")
```



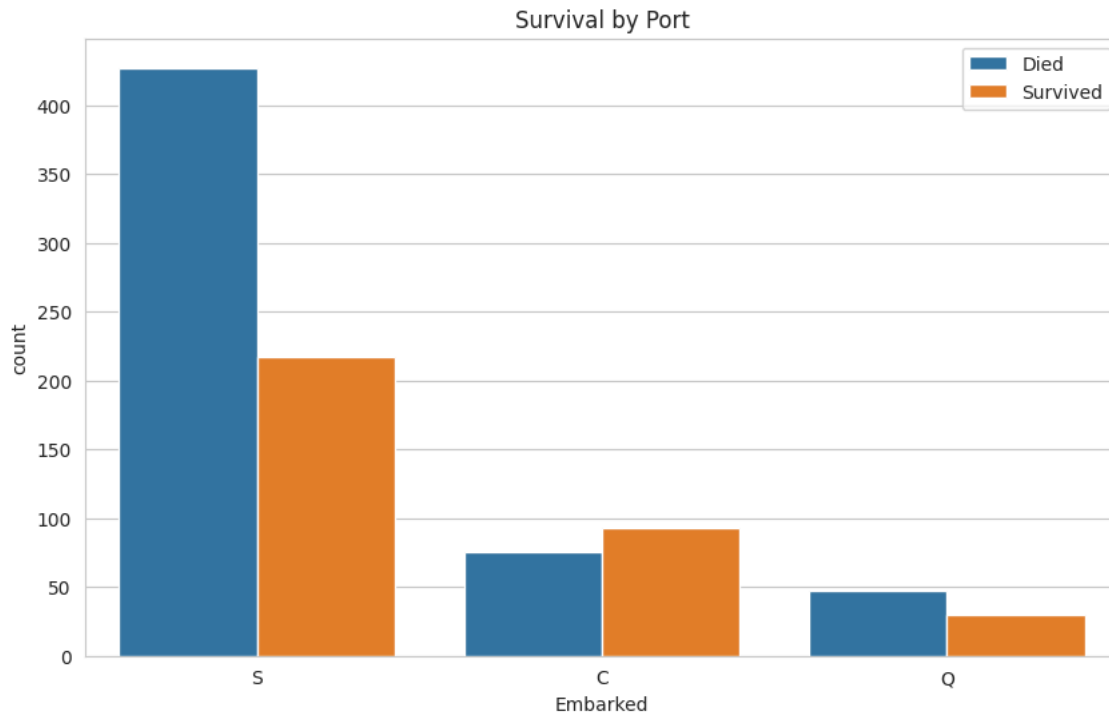
Insight: The age distribution for those who did not survive is centered around the 20-40 age range. For those who survived, there is a wider distribution centered around the 20-40 age range.

- Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C =

- Cherbourg, Q = Queenstown, S = Southampton).2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

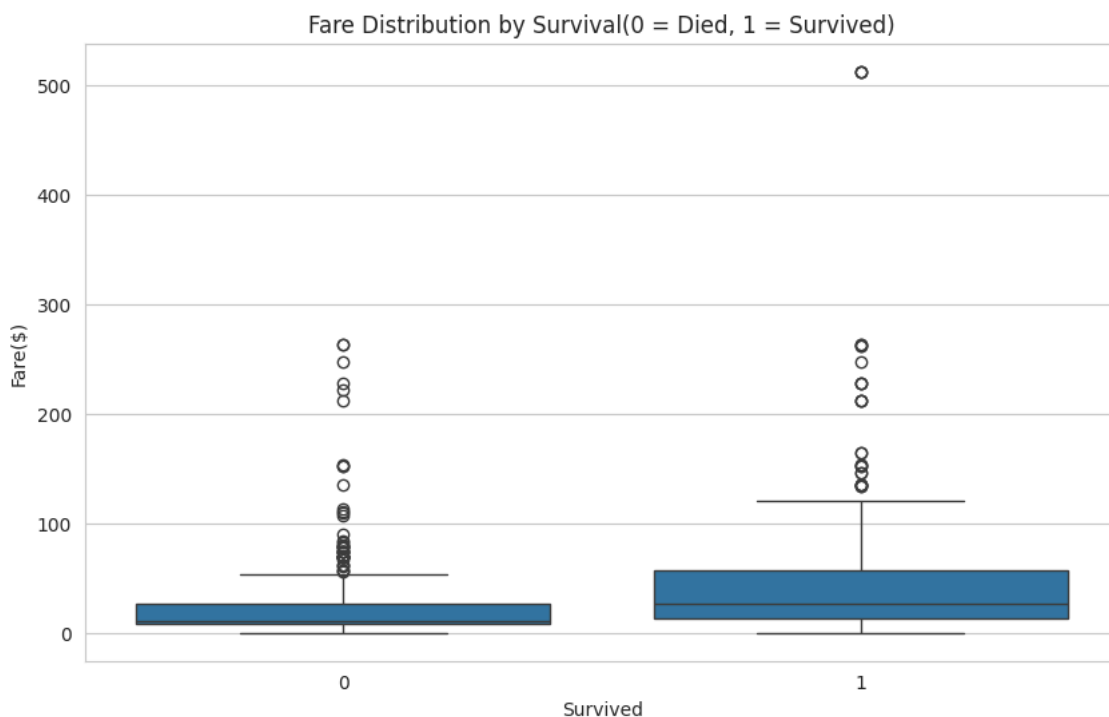
```
plt.figure(figsize=(10,6))
#Shows a countplot where person embarked and if they survived or not
sns.countplot(x = 'Embarked', hue='Survived', data=df)
plt.title('Survival by Port')
plt.legend(['Died', 'Survived'])
plt.show()
print("Southampton had the most amount of boarding and deaths being double the survived. Cherbourg was the only one with more li")
```



Southampton had the most amount of boarding and deaths being double the survived. Cherbourg was the only one with more living than

- Experiment 2: Fare vs. Survival1. Fare is a continuous numerical variable.2. A boxplot or violinplot is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the Fare distribution by Survived. What does it tell you?

```
plt.figure(figsize=(10, 6))
#Shows a box plot of who survived by fare
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare Distribution by Survival(0 = Died, 1 = Survived)')
plt.xlabel('Survived')
plt.ylabel('Fare($)'')
plt.show()
print("It shows me that on average the people that spent more money were more likely to survive the titanic")
```





Knowledge CheckInstructions: Answer the following questions in this markdown cell.1.

****What is the primary goal of Exploratory Data Analysis (EDA)?2. Based on the plots in this lab, what kind of person had the best chance of surviving the Titanic? (Describe them in terms of class, gender, and age).3. Why is it important to visualize data instead of just looking at summary statistics? What can a plot show you that a number like 'mean' or 'count' can't?**

1. The primary goal of EDA is to help data scientists understand the main idea of the data. It also helps find any outliers or missing values.
2. You had the best chance of survival if you were a female child, and also a passenger in class one.
3. It's important to visualize data instead of just a summary because it can show you outliers that you can't see with the mean, as it can also throw off the mean, so visualization can help you see the shape of the graph across groups.